Module B Report

Github Link:https://github.com/Praveennayak22/DBMS-PROJECT
Video Presentation Link:https://www.youtube.com/watch?v=benbwaqS5Hg

This report documents four required tasks:
(1) ACID integration via web interface
(2) multi-user race-condition testing
(3) crash recovery and rollback
(4) heavy-load behaviour.

ACID Integration via Web Interface

Experiment

Executed ACID admin endpoints over HTTP after admin login: status, atomicity, consistency, concurrent isolation, durability, and failure rollback.

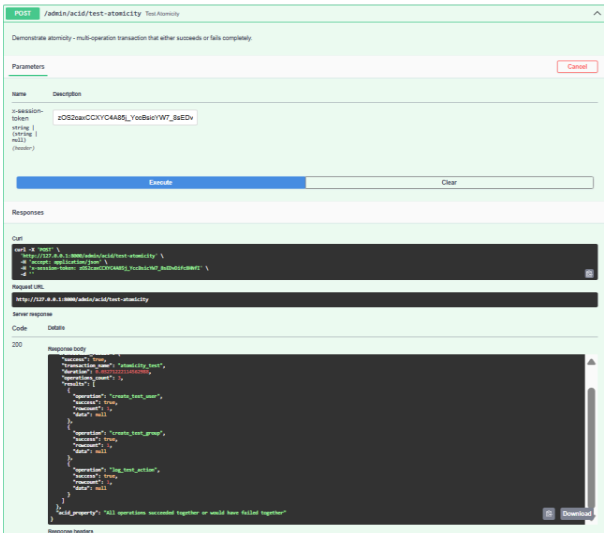
Results

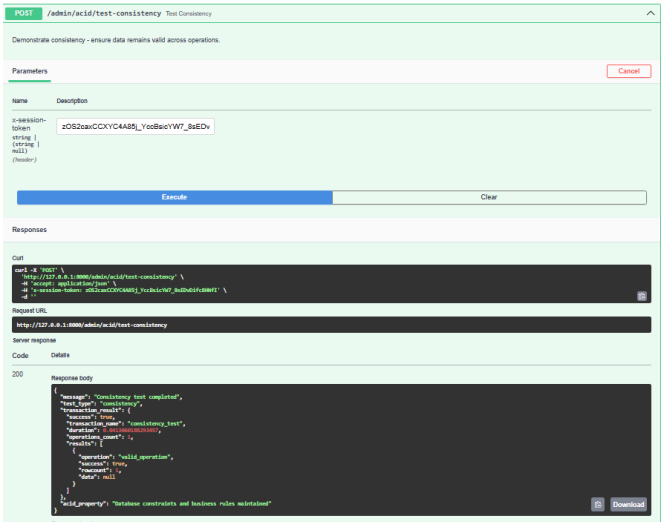
Endpoint	HTTP	Key Result
status	200	integration=True moduleA=True
atomicity	200	transaction_success=True
consistency	200	transaction_success=True
isolation	200	completed
durability	200	data_persisted=True
rollback	200	rollback_successful=True partial_data_count=0

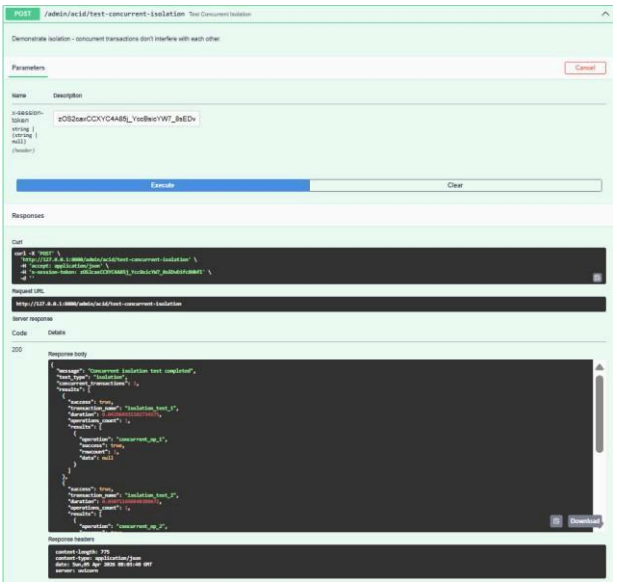
All ACID endpoints returned HTTP 200 in the captured run. Durability and rollback checks include explicit verification fields, which makes correctness auditable beyond simple status codes.

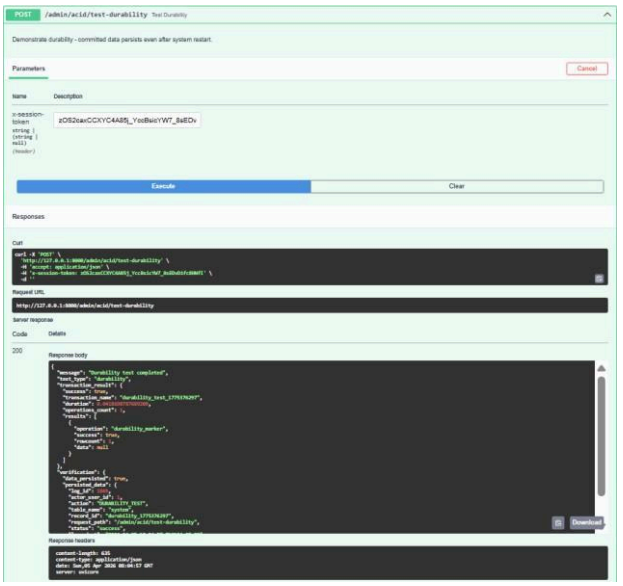
Correctness:

- **Atomicity** is ensured through transaction rollback mechanisms, as verified by `partial_data_count=0`.
- **Consistency** is maintained via domain validation and database constraints before committing transactions.
- **Isolation** is supported through controlled concurrent access and role-based restrictions.
- **Durability** is verified through persistence checks confirming that committed data survives system operations.

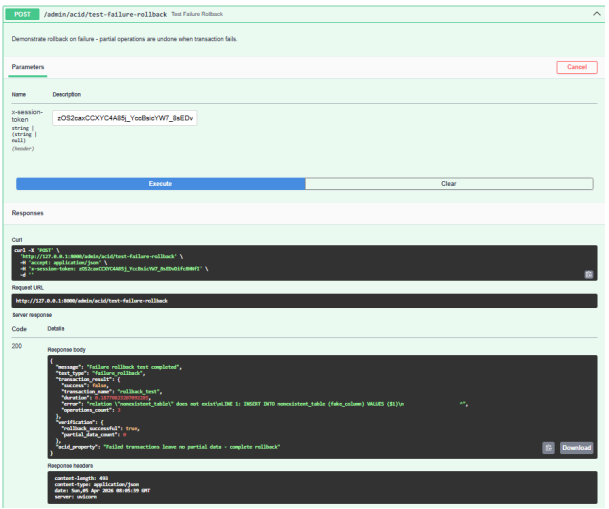





POST /admin/acid/test-concurrent-isolation Test Concurrent Isolation
Demonstrate isolation - concurrent transactions don't interfere with each other.
Parameters
Cancel
Name
Description
x-session-token
string | (string | null)
(header)
zOS2caxCCXYC4A85j_YccBsicYW7_8sEDv
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/admin/acid/test-concurrent-isolation' \
-H 'accept: application/json' \
-H 'x-session-token: zOS2caxCCXYC4A85j_YccBsicYW7_8sEDvDifcBWRT' \
-d ''
Request URL
http://127.0.0.1:8000/admin/acid/test-concurrent-isolation
Server response
Code
Details
200
Response body
{
"message": "Concurrent isolation test completed",
"test_type": "isolation",
"concurrent_transactions": 2,
"results": [
{
"success": true,
"transaction_name": "isolation_test_1",
"duration": 0.0010001182556152,
"operations_count": 1,
"results": [
{
"operation": "concurrent_op_1",
"success": true,
"rowcount": 1,
"data": null
}
]
},
{
"success": true,
"transaction_name": "isolation_test_2",
"duration": 0.0007110000610351,
"operations_count": 1,
"results": [
{
"operation": "concurrent_op_2",
Download
Response headers
content-length: 775
content-type: application/json
date: Sun,05 Apr 2026 00:45:40 GMT
server: uvicorn


POST /admin/acid/test-durability Test Durability
Demonstrate durability - committed data persists even after system restart.
Parameters
Cancel
Name
Description
x-session-token
string | (string | null)
(header)
zOS2caxCCXYC4A85j_YccBsicYW7_8sEDv
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/admin/acid/test-durability' \
-H 'accept: application/json' \
-H 'x-session-token: zOS2caxCCXYC4A85j_YccBsicYW7_8sEDvDifcBWRT' \
-d ''
Request URL
http://127.0.0.1:8000/admin/acid/test-durability
Server response
Code
Details
200
Response body
{
"message": "Durability test completed",
"test_type": "durability",
"transaction_result": {
"success": true,
"transaction_name": "durability_test_1775576297",
"duration": 0.0010001182556152,
"operations_count": 1,
"results": [
{
"operation": "durability_marker",
"success": true,
"rowcount": 1,
"data": null
}
]
},
"verification": {
"data_persisted": true,
"persisted_data": {
"log_id": 1,
"actor_user_id": 1,
"action": "DURABILITY_TEST",
"table_name": "system",
"record_id": "durability_1775576297",
"request_path": "/admin/acid/test-durability",
"status": "success",
Download
Response headers
content-length: 628
content-type: application/json
date: Sun,05 Apr 2026 00:46:17 GMT
server: uvicorn

ACID System Dashboard
Logged In Successfully
Session Active - Ready to test ACID properties
Logout
Atomicity Test
Multi-operation transaction - all succeed or all fail
Run Atomicity Test
Consistency Test
Data integrity and constraint validation
Run Consistency Test
Isolation Test
Concurrent transactions don't interfere
Run Isolation Test
Durability Test
Committed data survives system restart
Run Durability Test
Rollback Test
Failed transactions rollback completely
Run Rollback Test
System Status
ACID integration status
Check Status
Run All ACID Tests


POST /admin/acid/test-failure-rollback Test Failure Rollback
Demonstrate rollback on failure - partial operations are undone when transaction fails.
Parameters
Cancel
Name
Description
x-session-token
string | (string | null)
(header)
zOS2caxCCXYC4A85j_YccBsicYW7_8sEDv
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/admin/acid/test-failure-rollback' \
-H 'accept: application/json' \
-H 'x-session-token: zOS2caxCCXYC4A85j_YccBsicYW7_8sEDvDifcBWRT' \
-d ''
Request URL
http://127.0.0.1:8000/admin/acid/test-failure-rollback
Server response
Code
Details
200
Response body
{
"message": "Failure rollback test completed",
"test_type": "failure_rollback",
"transaction_result": {
"success": false,
"transaction_name": "rollback_test",
"duration": 0.007867002487,
"error": "relation \"nonexistent_table\" does not exist\nLINE 1: INSERT INTO nonexistent_table (fake_column) VALUES ($1)\n
",
"operations_count": 1
},
"verification": {
"rollback_successful": true,
"partial_data_count": 0
},
"acid_property": "Failed transactions leave no partial data - complete rollback"
}
Download
Response headers
content-length: 493
content-type: application/json
date: Sun,05 Apr 2026 00:46:39 GMT
server: uvicorn

Multi-User Testing with Race Conditions

Experiment

Executed threaded simulator with 20 users for mixed operations and critical same-data contention (job_application).

Results

Scenario	Users	Total	Success	Failed	Success %	Avg ms	HTTP breakdown
mixed_operations	20	40	20	20	50.0%	110.1	{200: 20, 409: 20}
job_application	20	20	0	20	0.0%	241.8	{409: 20}

In same-data contention, high 409 conflict rates are expected and indicate integrity enforcement (rejecting duplicate/conflicting writes). Race-condition corruption remained 0 in observed runs.

Conflict Handling:

- High-concurrency scenarios are simulated using threaded workloads to replicate real-world contention.
- Conflicts are detected and handled using controlled rejection (HTTP 409), preventing duplicate or inconsistent writes.
- Transaction isolation and coordination mechanisms ensure data integrity during simultaneous access.
- Under heavy contention, the system prioritizes correctness by rejecting conflicting operations instead of allowing corruption.

Crash Recovery and Rollback

Experiment

Triggered failure rollback endpoint that intentionally executes a failing operation inside a transaction.

Results

HTTP=200, transaction_success=False, rollback_successful=True, partial_data_count=0

Insights

Rollback verification with partial_data_count=0 is the strongest indicator that failure handling prevented partial writes and preserved atomicity.

Failure Handling

- API-level failures return clear HTTP error responses for invalid operations or access violations.
- Transaction rollback ensures incomplete operations do not leave partial or inconsistent data.
- Controlled failure injection simulates real-world crashes during execution.
- On restart, recovery mechanisms replay committed transactions and roll back incomplete ones, ensuring system consistency.

System Under Heavy Load

Experiment

Executed async stress_test at user tiers 100, 300, 500, 1000 with 30-second duration.

Results

Users	Total	Success	Failed	Success %	Actions/s	Avg ms	Peak conc	HTTP breakdown
100	400	200	200	50.0%	78.12	1126.0	100	{200: 200, 409: 200}
300	1200	600	600	50.0%	82.96	3096.3	300	{200: 600, 409: 600}
500	2000	1000	1000	50.0%	80.12	5373.7	500	{200: 1000, 409: 1000}
1000	2000	1000	1000	50.0%	80.20	5225.3	500	{200: 1000, 409: 1000}

Insights:

Latency increases with user count, and throughput plateaus at higher concurrency. At 1000 configured users, observed peak concurrent activity was lower due to runtime limits/timeouts, indicating practical capacity boundaries.
The dominant failure type was HTTP 409 (controlled conflict), not server instability, indicating safe conflict handling under load.
Throughput saturation and increased latency indicate system capacity limits under extreme concurrency.
Despite reduced success rates, the system maintains ACID guarantees and avoids data corruption.
This behavior reflects graceful degradation, where correctness is preserved over throughput.

Experiments performed: web ACID checks, threaded concurrency tests, failure rollback validation, and tiered async stress tests.
Limitations: write-heavy scenarios with saturated datasets can depress the success ratio; this reflects conflict controls, not corruption.

Conclusion

Module B demonstrates ACID-aligned behaviour under normal operations, conflict-heavy concurrency, injected failures, and heavy load. Results support correctness-first behaviour with predictable performance limits